Security Testing: Identifying Vulnerabilities in AI-Generated Code

NAME:U.V.Manohar Reddy

ROLL.NO:2503A52397

BATCH-14

Task 1– Input Validation Check Task:

Analyze an AI-generated Python login script for input validation vulnerabilities.

Instructions:

Prompt AI to generate a simple username-password login program.

Review whether input sanitization and validation are implemented.

Suggest secure improvements (e.g., using re for input validation).

Expected Output:

A secure version of the login script with proper input validation.

PROMPT:

Generate a Python username-password login program.
Check whether username and password inputs are valid.

Use regular expressions to validate the username.

Improve the code with proper input validation.

CODE:

```
import re

def validate_username(username):
    pattern = r"^[a-zA-Z0-9_]{4,15}$"
    return re.fullmatch(pattern, username) is not None


def validate_password(password):
    return len(password)>=8

username = input("Enter username: ")

password = input("Enter password: ")

if not validate_username(username):
    print("Invalid username!")
elif not validate_password(password):
    print("Password must contain at least 8 characters!")
else:
```

```
print("Login input validation successful!")
```

OUTPUT:

```
PS C:\Users\manvi\OneDrive\Desktop\AI_17> &  C:\Users\manvi\AppData\Local\Programs\Python\Python314\python.exe
'c:/Users/manvi/OneDrive/Desktop/AI_17/task-1.py'
Enter username: manvith
Enter password: manvith123
Password must contain at least 8 characters!
```

Task 2– SQL Injection Prevention Task: Test an AI-generated script that performs SQL queries on a database.

Instructions:

Ask AI to generate a Python script using SQLite/MySQL to fetch user details.

Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).

Refactor using parameterized queries (prepared statements).

Expected Output:

A secure database query script resistant to SQL injection.

PROMPT:

Generate a Python SQLite program to fetch user details.
Identify SQL injection risks in the database query.
Use parameterized queries instead of string concatenation.
Provide a secure and corrected version.

CODE:

```
import sqlite3


conn = sqlite3.connect("users.db")
cursor = conn.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY,
    username TEXT,
    email TEXT
)
""")

cursor.execute(
    "INSERT INTO users (username, email) VALUES (?, ?)",
    ("Mani123", "mani@example.com")
)
conn.commit()


username = input("Enter username: ")
```

```
# Secure parameterized query
cursor.execute(
    "SELECT * FROM users WHERE username = ?",
    (username,)
)

result = cursor.fetchone()

if result:
    print("User Details:", result)
else:
    print("User not found")

conn.close()
```

OUTPUT:

```
ython.exe' 'c:/Users/manid/OneDrive/Desktop/AI_17/task2_.py'
Enter username: mani123
User not found
PS C:\Users\manid\OneDrive\Desktop\AI_17>
```

Task 3– Cross-Site Scripting (XSS) Check Task:

Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.

Instructions:

Ask AI to generate a feedback form with JavaScript-based output.

Test whether untrusted inputs are directly rendered without escaping.

Implement secure measures (e.g., escaping HTML entities, using CSP).

Expected Output:

A secure form that prevents XSS atta

PROMPT:

Generate an HTML feedback form using JavaScript.
Display user feedback on the webpage.
Identify possible XSS vulnerabilities in the code.
Prevent XSS using textContent and secure coding practices.

CODE:

```
<!DOCTYPE html>
<html>
<head>
    <title>Feedback Form</title>
```

```
    <meta http-equiv="Content-Security-Policy"
        content="default-src 'self'; script-src 'unsafe-inline';">
</head>
<body>

    <h2>Feedback Form</h2>

    <input type="text" id="feedback"
        placeholder="Enter feedback">

    <button onclick="submitFeedback()">Submit</button>

    <h3>Your Feedback:</h3>
    <p id="output"></p>

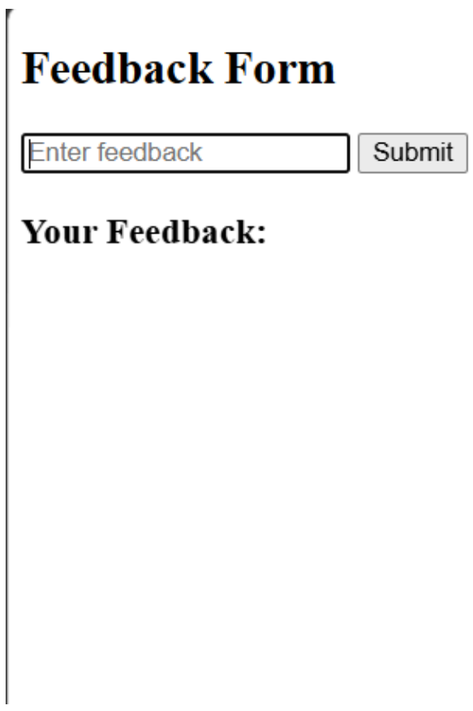
    <script>
        function submitFeedback() {
            let feedback =
                document.getElementById("feedback").value;
```

```
    document.getElementById("output").textContent =
        feedback;
}
    </script>


</body>

</html>
```

OUTPUT:



Task 4 – Real-Time Application:

Security Audit of AI-Generated Code Scenario: Students pick an AI-generated project snippet (e.g., login form, API integration, or file upload).

Instructions:

Perform a security audit to detect possible vulnerabilities.

Prompt AI to suggest secure coding practices to fix issues.

Compare insecure vs secure versions side by side.

Expected Output:

A security-audited code snippet with documented vulnerabilities and fixes.

PROMPT:

Generate a Python application that fetches user details from SQLite.
Perform a security audit and identify possible vulnerabilities.
Suggest secure coding practices to fix the issues.
Compare the insecure and secure versions of the code.

CODE:

```
import sqlite3

import re


conn = sqlite3.connect("users.db")

cursor = conn.cursor()


cursor.execute("""

CREATE TABLE IF NOT EXISTS users (

    id INTEGER PRIMARY KEY,
```

```
    username TEXT,
    email TEXT
)
""")

cursor.execute(
    "INSERT INTO users (username, email) VALUES (?, ?)",
    ("Mani123", "mani@example.com")
)
conn.commit()

username = input("Enter username: ")

# Input validation
if re.fullmatch(r"[A-Za-z0-9_]{4,15}", username):

    # Secure SQL query
    cursor.execute(
        "SELECT * FROM users WHERE username = ?",
        (username,)
```

```
)


result = cursor.fetchall()

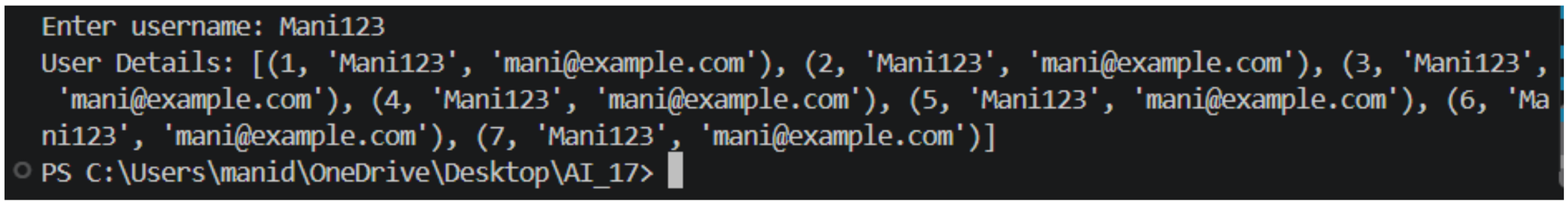
print("User Details:", result)


else:

    print("Invalid username!")


conn.close()
```

OUTPUT:



Task 5– Insecure Data Serialization Check Task: Evaluate AI-generated code that uses pickle for saving user data.

Instructions:

Ask AI to generate a script using pickle.load().

Explain why untrusted pickle data is dangerous. Replace with safer formats like JSON.

Expected Output:

Secure serialization code using JSON instead of pickle.

PROMPT:

Generate a Python program to save and load student data using pickle.
Identify the security risks of using pickle.load().
Replace pickle with JSON serialization.
Provide a secure version of the program.

CODE:

```
import json


student = {
    "name": "Mani",
    "roll_no": "2503A52388",
    "branch": "AIML"
}


# Save data using JSON
with open("student.json", "w") as file:
    json.dump(student, file, indent=4)


print("Data saved successfully!")
```

```
# Load data using JSON
with open("student.json", "r") as file:
    data = json.load(file)

print("Student Details:")
print(data)
```

OUTPUT:

